NEXO: INTERFACE HUMANO-MÁQUINA PARA MONITORAMENTO REMOTO DE LABORATÓRIOS DIDÁTICOS

NEXO: HUMAN-MACHINE INTERFACE FOR REMOTE MONITORING OF EDUCATIONAL LABORATORIES

LADOSKY, Vinicius[1]; MARQUES, Matheus[1]; VELOSO, Amanda[1]; CARVALHO, Leandro[2];
[1]Graduando(a) do Curso de Engenharia da Computação da Universidade São Francisco;
[2]Professor do Curso de Engenharia da Computação da Universidade São Francisco

vinicius.ladosky@mail.usf.edu.br / matheus.caldeira@mail.usf.edu.br
amanda.veloso@mail.usf.edu.br

RESUMO. Para realização de experimentos para pesquisa e coletas de dados em laboratórios didáticos, é necessário um forte controle do tempo, uma vez que a disponibilidade é limitada aos horários livres desses espaços à disposição dos alunos e professores. No entanto, muitos projetos e estudos não podem ser evoluídos de forma limitada. As disciplinas dos cursos de engenharia que envolvem sistemas embarcados e automação dependem de atividades práticas, mas muitos alunos enfrentam dificuldades para acompanhar os experimentos fora do ambiente de laboratório, comprometendo o aprendizado e limitando o desenvolvimento de habilidades técnicas. A proposta amplia um projeto anterior já desenvolvido, incorporando novas funcionalidades, com o objetivo de permitir que os alunos tenham acesso ao laboratório de forma remota viabilizando o acompanhamento de projetos mais longos. Nesse sentido, o trabalho explora o uso de tecnologia de baixo custo para mitigar o problema. Mais especificamente, discorre a implementação de um sistema composto de interface gráfica para navegadores de internet, servidor e dispositivo embarcado que automatiza o processo de criação de ambientes para interação e monitoramento de equipamentos de laboratórios remotamente. O sistema permite que usuários se identifiquem e criem salas virtuais para compartilhamento com outros usuários, permitindo simular a experiência da sala de aula. A ferramenta para criação de ambientes virtuais busca ser intuitiva para que mesmo professores sem afinidade com a tecnologia possam usá-la, além de prezar pela generalidade a fim de atender o maior número possível de casos de uso.

Palavras-chave: laboratório remoto, *MQTT*, transmissão de vídeo.

ABSTRACT. To conduct experiments for research and data collection in educational laboratories, strict time management is required, as access is limited to the availability of these spaces for students and teachers. However, many projects and studies cannot progress under such restricted conditions. Engineering courses that involve embedded systems and automation rely heavily on practical activities, yet many students face difficulties following experiments outside the laboratory environment, compromising learning and limiting the development of technical skills. This work extends a previously developed project by incorporating new functionalities aimed at enabling students to access the laboratory remotely, thus facilitating the monitoring of longer-term projects. In this context, the study explores the use of low-cost technology to mitigate the problem. More specifically, it describes the implementation of a

system composed of a web-based graphical interface, a server, and an embedded device that automates the creation of environments for remote interaction and laboratory equipment monitoring. The system allows users to log in and create virtual rooms to share with other users, simulating a classroom experience. The tool for creating virtual environments is designed to be intuitive so that even teachers with limited technological skills can use it, while also emphasizing generality to accommodate the widest possible range of use cases.

Keywords: remote laboratory, *MQTT*, video streaming.

INTRODUÇÃO

Em laboratórios, é comum que certos experimentos levem longos períodos para serem realizados. Em ambientes comerciais, isso é facilitado por automação, possibilitando que sejam realizados sem supervisão continuamente (BROWN e BADRICK, 2023). Isso torna viável a criação de ampla base de dados, possibilitando análises estatísticas mais substanciais. Por outro lado, laboratórios didáticos em ambientes acadêmicos muitas vezes enfrentam dificuldades para replicar esse tipo de experimento, devido à falta de recursos e equipamentos de automação e monitoramento remoto (BERALDO, OLIVEIRA e STRINGHINI, 2021).

Assim, este estudo é direcionado a laboratórios didáticos de baixo custo, discutindo a implementação de uma solução para automação de experimentos. O projeto engloba a criação de interface gráfica para navegadores de forma a possibilitar interação e monitoramento remotos em tempo real, infraestrutura de servidor para armazenamento dos dados coletados em experimentos e o uso de dispositivo embarcado para manipular os equipamentos do laboratório e coletar amostras dos sensores. É esperado que o sistema implementado possa ser utilizado mesmo por pessoas com menor grau de afinidade com tecnologia, com interface intuitiva e acessível.

Soluções similares são comumente denominadas de “laboratórios remotos” - em contrapartida aos “laboratórios virtuais”. A primeira consiste em instalar câmeras, sensores, atuadores e um microcontrolador em um laboratório e prover acesso aos equipamentos de forma remota. Já a segunda consiste em simular computacionalmente o comportamento dos equipamentos, sem que estejam de fato instalados em um local físico. Ambas as vertentes já vêm sendo estudadas há mais de duas décadas, com diversos estudos validando sua viabilidade técnica e econômica (GOLINELLI, SILVA e YEVSEYEVA, 2018), bem como sua relevância como ferramenta de apoio ao ensino (BERALDO, OLIVEIRA e STRINGHINI, 2021).

A escolha do tema decorre do intuito de estender a pesquisa de iniciação científica “Análise e Processamento Inteligente de Imagens em Laboratórios Remotos”, em que é desenvolvida uma plataforma para interação remota com um experimento de coluna de destilação. O projeto envolve o uso de um *Raspberry Pi* para coletar dados dos sensores e mover os atuadores. O embarcado se comunica via protocolo *Message Queuing Telemetry Transport* (*MQTT*) com a infraestrutura de servidores para permitir monitoramento e interação em tempo real com o experimento através da interface gráfica desenvolvida para navegadores de *Internet*.

No laboratório, é também posicionada uma câmera de forma a capturar a bancada do experimento, que tem suas imagens enviadas via protocolo *Real Time Messaging Protocol Safe* (*RTMPS*) para visualização em tempo real.

Este trabalho pretende modificar a solução implementada na pesquisa mencionada para que seja aplicável a laboratórios didáticos em geral, com enfoque em acessibilidade e automação de experimentos de longa duração, permitindo a coleta de amplas bases de dados para ambientes acadêmicos de baixo custo. Assim, a fim de reduzir custos, será utilizado o *ESP-32*, ao invés do *Raspberry Pi*. Outras tecnologias utilizadas serão mantidas, como canais de transmissão de vídeo em tempo real *Interactive Video Service* (*IVS*) provido pela *Amazon Web Services* (*AWS*), *MQTT, MySQL* e Angular. A captura de imagens e transmissão será realizada com aplicativo de terceiros, a critério do usuário final.

A escolha dessas tecnologias em específico não é arbitrária, sendo as mais comumente utilizadas em outros estudos sobre o assunto que obtiveram resultados positivos. O protocolo *MQTT*, por exemplo, é uma tecnologia pensada especificamente para comunicação direta entre máquinas, sendo capaz de economizar consumo energético devido ao tamanho reduzido dos pacotes *IP* (*Internet Protocol*) de até 2 *bytes*, além de proporcionar versatilidade quanto ao formato das mensagens e compatibilidade com uma ampla gama de dispositivos embarcados (ATMOKO, 2017). De forma semelhante, a escolha do banco de dados *MySQL* também é uma das mais comuns nesses projetos como relatado (BERALDO, OLIVEIRA e STRINGHINI, 2021), sendo provido muitas vezes de forma gratuita ou com custo reduzido, possibilitando a criação de sistemas de autenticação e integração com serviços de terceiros como a *AWS*.

LEVANTAMENTO BIBLIOGRÁFICO

Interactive Video Service (*IVS*)

Esse serviço provido pela *AWS* abstrai a complexidade envolvida na transmissão de vídeo em tempo real incluindo captura, ingestão, processamento, entrega e, finalmente, reprodução (AMAZON WEB SERVICES, 2025). Sua utilização envolve a criação de canais de comunicação para cada *stream*, sendo possível especificar configurações como qualidade de vídeo e latência. Esses canais podem ser criados manualmente através da interface de usuário, interface de linha de comando ou programaticamente via *Software Development Kit* (*SDK*) da *AWS*. Assim, são disponibilizados *URL* de reprodução e transmissão para utilização posteriormente, sendo válidos enquanto o canal não for excluído.

A ingestão de vídeo é feita com protocolo *RTMPS*, que mantém internamente uma conexão persistente em *TCP*, possibilitando comunicação de baixa latência. Para isso, o vídeo é dividido em fragmentos de tamanho variável negociado entre cliente e servidor.

Com um canal criado, é feita a captura de imagens utilizando algum cliente como aplicativo de terceiros compatível com o protocolo *RTMPS* ou o *SDK* de *broadcast*

disponibilizado pela *AWS*, que deve ser configurado para se conectar ao *URL* de transmissão recebido (AMAZON WEB SERVICES, 2025).

O vídeo pode ser reproduzido com o *IVS Player SDK*, disponível para aplicações *web*, *iOS* e *Android*, que implementa funcionalidades essenciais como controle de *buffer* para reduzir atraso e alternância dinâmica de qualidade de vídeo (AMAZON WEB SERVICES, 2025). No caso de aplicações *web* como é o desse projeto, é criada uma instância de um cliente com o *IVS Player SDK* que é, então, associado a um elemento *HTML* de vídeo na página e conectado a um *URL* de reprodução disponibilizado pelo *IVS*.

TypeScript e Transpilação

O *TypeScript* é uma linguagem de programação que estende a linguagem *JavaScript* adicionando suporte à definição de tipos às variáveis (TYPESCRIPT, 2025). Para isso, é utilizado um programa especial denominado transpilador, que converte o código escrito em *TypeScript* para *JavaScript* equivalente, checando por inconsistências de tipagem. Esse processo é chamado de transpilação em alusão ao processo de compilação, que, de maneira similar, converte código em linguagem de alto nível para linguagem de montagem e, também de maneira similar, introduz artificialmente conceitos de alto nível como tipos e estruturas de controle.

Da perspectiva do programador, o transpilador *TypeScript* é uma forma de delegar a checagem de inconsistências de tipagem, o que, no longo prazo, acelera o processo de desenvolvimento e aumenta a confiabilidade do código, possibilitando detecção de erros mesmo antes que haja qualquer fluxo de execução, ainda na fase de transpilação. Esse recurso se torna mais relevante à medida que cresce a complexidade do código. Assim, dado o escopo do projeto e expectativa de complexidade de implementação no mínimo moderada, foi decidido pelo uso dessa ferramenta.

Message Queuing Telemetry Transport (MQTT)

Se trata de um protocolo de comunicação leve e eficiente, amplamente adotado em aplicações de Internet das Coisas (*IoT*) devido à sua baixa sobrecarga e simplicidade de implementação. Baseado no modelo publicador/assinante, ele promove o desacoplamento entre emissores e receptores por meio de um *broker*, responsável por gerenciar e encaminhar mensagens entre os dispositivos conectados (DA SILVEIRA, AMICI, *et al.*, 2021). Operando sobre o protocolo *TCP*/*IP*, garante confiabilidade na transmissão de dados e oferece três níveis de Qualidade de Serviço (*QoS*) que se adaptam às necessidades específicas de cada aplicação (ZHOU, WAN, *et al.*, 2022). Entre suas funcionalidades estão o suporte à retenção de mensagens, ao chamado "*last will and testament*" e à persistência de sessões, tornando-o robusto mesmo em redes instáveis (LIGHT, 2017).

Neste projeto, sua principal utilização é no controle remoto dos diversos atuadores e sensores, na parte física da aplicação, tendo sido escolhido pela sua versatilidade e baixo custo, evidenciados pela utilização do protocolo em projetos de automação industrial, eficiência energética e controle ambiental em instituições de ensino por todo o país (LOPES e FERNANDES JÚNIOR, 2021).

ESP-32

O *ESP32* é um microcontrolador de baixo custo desenvolvido pela *Espressif Systems*, amplamente utilizado em aplicações de Internet das Coisas (*IoT*) devido à sua versatilidade e integração de recursos. Ele conta com processadores *Tensilica Xtensa LX6*, capazes de operar até 240 MHz, além de memória *SRAM* interna e suporte para memória *Flash* externa, oferecendo desempenho adequado tanto para tarefas simples de monitoramento quanto para aplicações mais complexas (ESPRESSIF SYSTEMS, 2020).

O dispositivo se destaca por integrar conectividade *Wi-Fi* e *Bluetooth* em um único *chip*, eliminando a necessidade de módulos adicionais e reduzindo custos de implementação. Outro diferencial está em seus modos de baixo consumo energético, como o *Deep Sleep*, que viabilizam seu uso em projetos alimentados por bateria e aplicações de longa duração (BRÖLL, HANSHANS e KIMMERLE, 2023).

A aplicação do *ESP32* neste projeto se justifica por apresentar modos de operação de baixo consumo energético e suporte a diferentes interfaces de comunicação, o que o torna apropriado para sistemas de coleta e transmissão contínua de dados. Além disso, sua capacidade de integrar *Wi-Fi* e *Bluetooth* em um único *chip* facilita a criação de soluções remotas com conectividade estável, eliminando a necessidade de módulos adicionais.

Estudos recentes também destacam sua viabilidade em cenários de ensino, pesquisa e prototipagem, demonstrando a adequação do microcontrolador em projetos de monitoramento e automação de baixo custo (CHEW e ARSHAD, 2021).

METODOLOGIA

A metodologia parte do princípio de dividir o sistema em sistemas menores, projetados para comunicar-se entre si com protocolos específicos. Isso facilita o desenvolvimento em equipe e testes de cada componente individualmente, diminuindo dificuldades ao integrar as partes. Assim, foram estipuladas três frentes de desenvolvimento: interface de usuário, infraestrutura de servidor e sistema embarcado; bem como protocolos de comunicação entre essas partes, como os caminhos disponibilizados pela *API* para interação com o servidor e a formatação das mensagens *MQTT*.

Protocolo de Mensagens MQTT

O protocolo *MQTT* permite a comunicação entre diferentes máquinas conectadas a um mesmo broker via mensagens em formato de texto ou até mesmo binário, não estabelecendo nenhuma imposição quanto ao conteúdo das mensagens. Portanto, faz-se necessário estabelecer protocolos específicos para cada contexto em que essa tecnologia é utilizada. Assim, para esse projeto, foram estipuladas determinadas regras e mecanismos para efetivar a comunicação entre a interface gráfica e o embarcado, estruturando as mensagens em formato *JSON* e dividindo-as nos tópicos "*status*", "*get*", "*set*", *"exp",* "*iniciar-exp*", "*parar-exp*" e "*sinc-exp*", prefixados com o identificador da sala virtual.

No tópico "*status*", são enviadas mensagens pelo embarcado com atualizações sobre eventuais alterações nas leituras dos sensores ou no estado dos atuadores. Os navegadores dos usuários inscrevem-se nesse tópico assim que se conectam a uma sala. As mensagens são recebidas em *JSON* seguindo um modelo pré-estabelecido, podendo conter o estado de todas ou algumas das variáveis.

Já nos tópicos "*get*" e "*set*", são publicadas mensagens pelos navegadores para requisitar o estado de uma ou todas as variáveis ou definir o valor dela caso refira-se a um atuador. Assim, o embarcado se inscreve nesses tópicos assim que possível e responde às mensagens no tópico "*get*" e "*set*" com o novo estado das variáveis correspondentes.

Finalmente, os tópicos "*iniciar-exp*", "*parar-exp*" e "*sinc-exp*" são utilizados para criar, finalizar e sincronizar experimentos, que se referem a um período em que o embarcado passa a coletar amostras periodicamente e registrá-las no banco de dados. Para cada experimento, o usuário pode definir um nome e período de amostragem desejado e acompanhar os resultados graficamente no *plotter* embutido na aplicação. O *plotter* acompanha um seletor que possibilita escolher qual experimento visualizar, podendo ser um experimento finalizado e registrado banco de dados ou um experimento ainda em andamento. No último caso, os resultados são atualizados em tempo real, mesclando dinamicamente consultas ao banco de dados e tratamento de mensagens *MQTT* recebidas no tópico "*exp*", a fim de equilibrar a carga entre o servidor e o embarcado e reduzir o atraso entre a coleta de uma amostra e sua exibição na interface de usuário.

Interface de Usuário

Tendo a acessibilidade como um dos focos centrais do projeto, a interface de usuário foi desenvolvida para navegadores de *internet*, permitindo que a aplicação possa ser acessada sem necessidade de instalação e compatível com uma ampla gama de dispositivos – desde computadores pessoais a dispositivos móveis.

Um dos desafios que essa escolha implica, no entanto, é a necessidade de desenvolver a lógica da aplicação em *JavaScript*, que funciona bem para projetos simples, mas que pode oferecer dificuldades para projetos de maior escala. Um dos principais problemas é a ausência

restrições aos tipos das variáveis, que dificulta a legibilidade do código além deixar o desenvolvedor mais suscetível a cometer erros e, consequentemente, comprometendo a confiabilidade da aplicação. Assim, foi decidido o uso da linguagem *TypeScript*, que se trata de um superconjunto da linguagem *JavaScript*, isto é, oferecendo todos os recursos da linguagem *JavaScript*, mas com camadas de segurança como verificação de tipos e valores nulos, delegando, em boa parte, essas preocupações ao transpilador *TypeScript*. Além disso, a linguagem permite facilmente trabalhar com módulos, dividindo o código em pequenas partes encapsuladas e mais fáceis de manter e abstrair, qualidades essenciais para projetos de maior escala.

O uso de *TypeScript*, como demonstrado, traz diversos benefícios, mas também traz algumas inconveniências. Uma delas é a necessidade de transpilação, que converte o código em *TypeScript* para *JavaScript*, que é a única linguagem que os navegadores de *internet* efetivamente entendem. Isso adiciona uma camada de complexidade ao processo de desenvolvimento, especialmente à depuração, necessitando que sejam configurados mecanismos de mapeamento de código transpilado. Por tanto, para contornar essas inconveniências, foi utilizado o *framework Angular*. Essa ferramenta configura um ambiente de desenvolvimento *TypeScript* pronto para uso, permitindo depuração e integrando outras ferramentas como *Vite*, que permite que as alterações no código sejam refletidas no navegador em tempo real, facilitando o processo de desenvolvimento. O *framework Angular* também traz outros benefícios além desses, introduzindo uma sintaxe aumentada de *HTML* que permite o uso de diretivas especiais para uso de laços e estruturas de controle, além de facilitar a definição de componentes gráficos reutilizáveis, contribuindo para o reaproveitamento de código e encapsulamento.

O *framework Angular*, além de prover diversas funcionalidades e facilidades, propõe determinada forma de estruturar o código, dividindo-o em partes responsáveis pela lógica dos componentes gráficos da interface e partes responsáveis pela lógica de negócios, denominados "serviços". Estruturar o código dessa forma permite aproveitar o mecanismo de gerenciamento de recursos e injeção de dependências fornecido pelo *framework*. Assim, foram criados serviços para abstrair a comunicação *MQTT*, as interações com o servidor e o gerenciamento do processo de autenticação de usuários.

Serviço MQTT

O serviço de comunicação *MQTT* é responsável por abstrair a comunicação *MQTT* fornecendo uma interface simples para utilização nos outros componentes da aplicação. Para isso, é utilizado internamente a biblioteca *MQTT.js*, que implementa o processo de conexão com o *broker*, bem como o protocolo de envio e recebimento de mensagens e a inscrição aos tópicos *MQTT*.

Assim, externamente, o serviço expõe um objeto contendo todas as variáveis do experimento, que é mantido atualizado automaticamente conforme são recebidas novas mensagens *MQTT*. Isso é feito com a inscrição em um tópico *MQTT* durante a inicialização do serviço e a atribuição de um tratador assíncrono de mensagens. Esse tratador interpreta as mensagens de texto recebidas como *JSON* e atribui os valores ao objeto para que seja mantido atualizado. Além do objeto contendo as variáveis do experimento, são fornecidas funções para enviar mensagens específicas ao embarcado, formatando-as no modelo esperado pelo embarcado de acordo com os argumentos passados, como identificador do controlador e seu novo estado. Chamadas à essas funções também colocam as respectivas variáveis do experimento em estado inválido, até que seja confirmado o recebimento da mensagem por parte do embarcado, possibilitando que seja provida ao usuário uma representação visual mais fidedigna do estado real do sistema.

Um dos recursos fornecidos pela aplicação durante a criação de salas é a definição de uma função de transformação de unidade para cada variável do experimento, para que possa ser convertida para exibição na interface gráfica ou para envio ao embarcado. Essa funcionalidade também é implementada na camada do serviço *MQTT*. Para isso, é mantido internamente um *WebWorker* que executa a função apropriada para converter a unidade da variável toda vez que é recebida uma atualização do embarcado no tratador de mensagens *MQTT* ou é feito um comando por meio das funções providenciadas para uso em outros componentes da aplicação.

Serviço de API

O serviço de *API*, por outro lado, se responsabiliza por abstrair a interface de programação de aplicação implementada para o servidor, fornecendo uma função para cada recurso da *API*. Internamente, essas funções invocam o *URL* apropriado do recurso formatando os argumentos passados no formato esperado, seja como parâmetros de *query string*, cabeçalhos ou corpo de requisição *HTTP*. O serviço também se encarrega de autenticar todas as requisições feitas passando o *token* provido pelo serviço de autenticação, além de lançar exceções de classe apropriada para eventuais mensagens de erro recebidas pelo servidor.

Serviço de Autenticação

O serviço de autenticação abstrai as operações de criação de conta e início e término de sessão de usuário por meio de funções de acesso público, bem como a manipulação do *token* de autenticação e o gerenciamento de seu armazenamento.

Assim, internamente as funções de manipulação de sessão de usuário fazem chamadas à *API* do servidor, responsável por criar usuários ou verificar senhas para nomes de usuário fornecido pela aplicação e retornar um *token JWT* em caso de operação bem-sucedida. Esse

token é então decodificado para extrair informações como nome de exibição e identificador do usuário e disponibilizá-las publicamente para outras classes.

As sessões de usuário duradouras são implementadas com o uso da interface *LocalStorage*, provida pelos navegadores de *internet*. Essa interface permite armazenar dados na memória local do navegador do usuário, associando-se uma chave de acesso à informação a ser armazenada em formato de texto. Assim, o serviço armazena o *token* de autenticação quando é feita uma operação de início de sessão ou criação de conta, apagando-o da memória em operação de término de sessão.

Servidor

A metodologia empregada no desenvolvimento do *backend* teve como base a criação de uma arquitetura robusta, segura e modular, utilizando a linguagem *TypeScript* em conjunto com o ambiente de execução *Node.js* e o sistema gerenciador de banco de dados *MySQL*. O objetivo principal foi fornecer uma estrutura de comunicação eficiente entre o servidor e a aplicação cliente, garantindo autenticação, controle de usuários, gerenciamento de salas virtuais e experimentos remotos.

O banco de dados foi composto por tabelas responsáveis por armazenar informações de usuários, salas, experimentos, amostras e parâmetros de configuração do ambiente. Cada tabela foi criada com chaves primárias e estrangeiras para assegurar a integridade referencial dos dados. Entre as principais entidades estão: *usuarios, salas, experimentos* e *amostras*, que se relacionam de maneira a permitir o gerenciamento completo das atividades laboratoriais remotas.

A camada de *backend* foi desenvolvida utilizando o *framework Express.js* com *TypeScript*, seguindo o paradigma modular e orientado a serviços. O foco esteve na manipulação de requisições *HTTP* e na comunicação com o banco de dados, empregando a biblioteca *MySQL2* para o mapeamento e persistência dos dados. Foram criados diferentes módulos para o sistema, entre eles: *signup.ts, login.ts, salas.ts* e *experimentos.ts*, cada um responsável por funcionalidades específicas dentro do fluxo da aplicação.

O módulo *signup.ts* foi responsável pela criação de novos usuários no sistema. Esse *endpoint* recebe, via requisição *POST*, os dados em formato *JSON* contendo nome, usuário e senha, e realiza validações quanto ao tamanho e à complexidade da senha, utilizando expressões regulares para verificar a presença de caracteres maiúsculos, minúsculos, dígitos e símbolos especiais. Após a validação, as informações são registradas no banco de dados e é gerado um *token JWT* (*JSON Web Token*), utilizado para autenticação segura nas demais rotas do sistema.

O módulo *login.ts* teve a função de autenticar usuários previamente cadastrados. A partir de uma requisição *POST* contendo as credenciais, o servidor realiza a verificação no banco de dados e, em caso de sucesso, retorna um *token JWT* contendo as informações essenciais do usuário. Esse *token* é essencial para o controle de acesso às demais funcionalidades, evitando requisições não autorizadas.

O arquivo *salas.ts* implementou o controle e a listagem das salas virtuais disponíveis para o usuário autenticado. O sistema utiliza requisições *GET* acompanhadas do *token* de autenticação e retorna os dados de cada sala pertencente ao usuário, incluindo informações sobre modelo, proprietário e parâmetros de conexão *MQTT*. Essa estrutura é fundamental para o gerenciamento dos ambientes de experimentação dentro do laboratório remoto.

O módulo *experimentos.ts* foi responsável pelo gerenciamento completo dos experimentos, permitindo a criação, leitura, atualização e exclusão de registros (operações *CRUD*). O *backend* foi projetado para lidar com diferentes métodos *HTTP — GET, POST, PATCH* e *DELETE* —, permitindo a manipulação eficiente dos dados experimentais. Cada operação requer a presença de um *token JWT* válido, assegurando que apenas usuários autenticados possam interagir com os recursos do sistema.

Durante o desenvolvimento, o tratamento de exceções e erros foi centralizado em *middlewares* e utilitários específicos, que contribuíram para a padronização das respostas e a segurança da aplicação. Foram também adicionados cabeçalhos *Cross-Origin Resource Sharing* (*CORS*), possibilitando a comunicação entre diferentes domínios sem comprometer a integridade e a segurança do sistema.

Por fim, a metodologia adotada possibilitou a implementação de um *backend* escalável e seguro, apto a atender as necessidades de um laboratório remoto. O uso do *TypeScript* e do *Node.js*, aliado à autenticação *JWT* e às boas práticas de desenvolvimento, garantiu um ambiente confiável, performático e de fácil manutenção, adequado à integração com interfaces *web* e aplicações de controle de experimentos em tempo real.

Banco de dados

O banco de dados do sistema foi modelado com o objetivo de organizar e gerenciar as informações provenientes das salas. O modelo adotado é relacional, estruturado de forma a garantir integridade referencial, escalabilidade e facilidade de integração com os módulos de controle e monitoramento do sistema. O modelo relacional é amplamente utilizado em aplicações distribuídas por permitir o controle eficiente de entidades e seus relacionamentos, mantendo a consistência dos dados (HEUSER, 2024). Assim, a estrutura do banco foi concebida com base em entidades principais (como *salas*, *usuários* e *experimentos*) e em tabelas auxiliares que descrevem as relações entre elas.

A tabela *usuarios* armazena os dados dos participantes do sistema, incluindo identificadores únicos, nomes e credenciais de acesso. Esses usuários podem atuar como proprietários ou participantes de salas, conforme definido na tabela relacional *salas_usuarios*. Esta, por sua vez, estabelece um relacionamento N:N entre as entidades *usuários* e *salas*, permitindo que um mesmo usuário participe de várias salas e que cada sala tenha múltiplos usuários associados. O campo *proprietario* identifica o responsável pela administração de cada ambiente virtual.

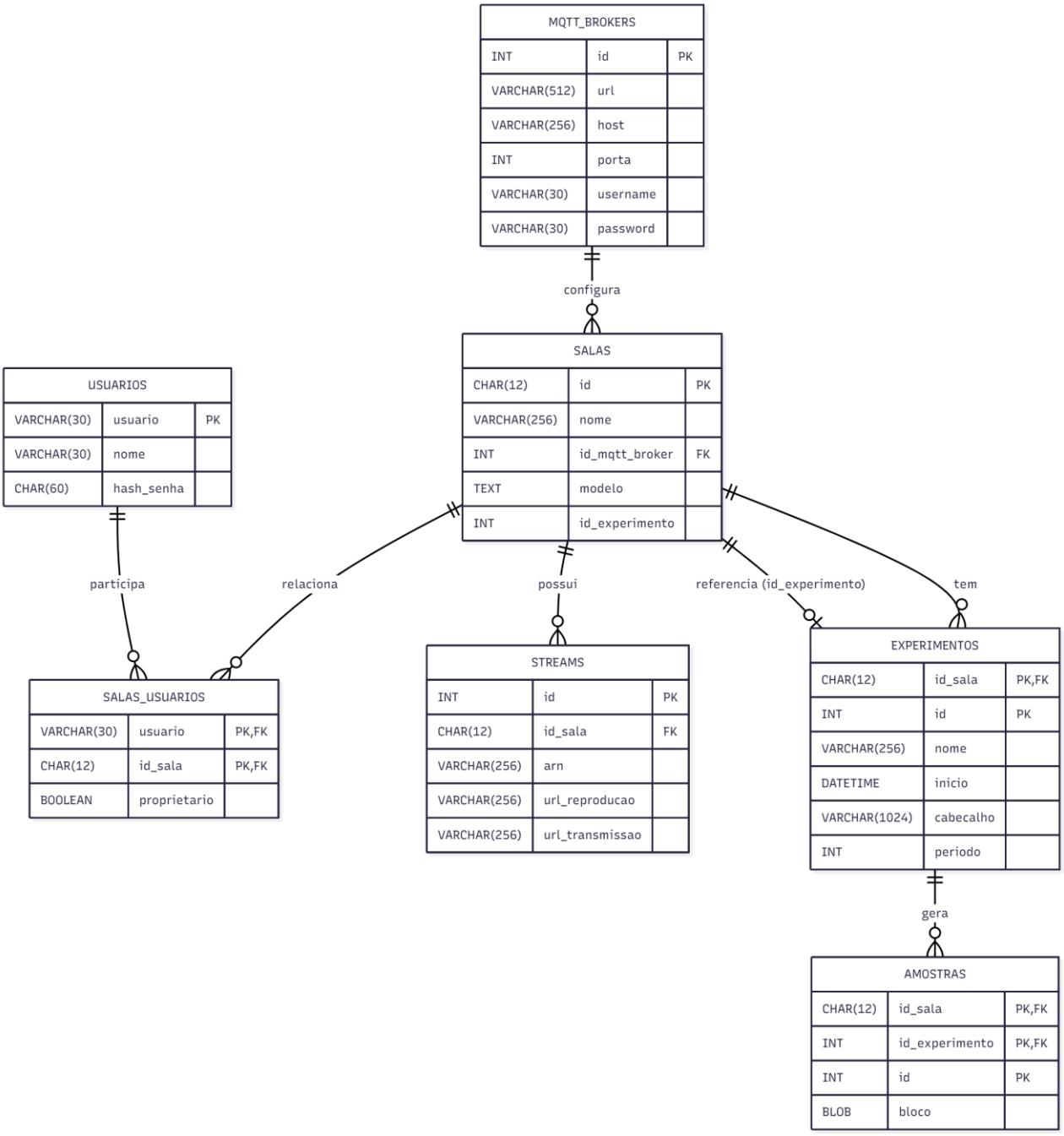
A tabela *salas* constitui o núcleo lógico do sistema. Ela representa os ambientes remotos e contém informações como nome, identificador do *broker MQTT* e um campo de referência para o experimento ativo. A inclusão da chave estrangeira *id_experimento* cria uma referência circular com a tabela experimentos, permitindo identificar qual experimento está em execução em determinada sala. Esse tipo de relação é útil em sistemas que necessitam monitorar processos em tempo real (DATE, 2004).

A tabela *mqtt_brokers* armazena as informações dos servidores responsáveis pela intermediação das mensagens enviadas e recebidas pelos dispositivos embarcados. Um mesmo *broker* pode atender a várias salas, configurando um relacionamento um-para-muitos (1:N) com a tabela *salas*. Essa estrutura favorece a escalabilidade e a descentralização da comunicação em modelos de sistemas distribuídos baseados em mensagens (SILBERSCHATZ, KORTH e SUDARSHAN, 2020).

A tabela *streams* contém os dados das transmissões de vídeo associadas às salas, incluindo as *URLs* de reprodução e transmissão e o identificador do recurso na nuvem (*ARN*). Cada sala pode possuir diversas transmissões, configurando outro relacionamento 1:N.

No contexto das atividades práticas e experimentais, a tabela *experimentos* armazena os registros de experimentos realizados em cada sala, identificados por uma chave composta (*id_sala, id*). Nela são guardadas informações como o nome do experimento, o horário de início, o cabeçalho dos dados coletados e o período de amostragem. As amostras coletadas são armazenadas na tabela *amostras*, a qual registra os dados brutos em formato binário, vinculando-se ao experimento por meio das chaves estrangeiras *id_sala* e *id_experimento*. Essa estrutura permite o registro eficiente de grandes volumes de dados experimentais, garantindo a integridade referencial.

Figura 1: Diagrama Entidade-Relacionamento do banco de dados



Fonte: Do próprio autor.

Conforme consta na Figura 1, o diagrama entidade-relacionamento (ER) representa essa estrutura e evidencia os relacionamentos centrais do sistema. A tabela salas aparece como elemento principal, conectando-se a diversas entidades. Essa modelagem garante modularidade e clareza na relação entre dispositivos físicos, usuários e experimentos.

Embarcado

O desenvolvimento do sistema embarcado teve como principal foco o baixo custo e uma aplicação robusta, garantindo um bom funcionamento e estabilidade mesmo em condições de conexão precária e baixa taxa de transmissão de dados. Para tal, foi essencial a escolha do *ESP32*, como um microcontrolador com boa capacidade de processamento, conexão nativa a *Wi-Fi* e suporte a diversos sistemas de arquivos e tecnologias de auxílio prático.

Seleção das tecnologias

Para fazer a comunicação e integrar os dispositivos do embarcado com o servidor e a interface, foi adotado o protocolo *MQTT* pela sua leveza e praticidade. A biblioteca escolhida para garantir seu funcionamento foi *MQTT.h* de GAEHWILER, 2024.

A autenticação e obtenção de informações das salas através das requisições *HTTP* foram implementadas com a biblioteca *HTTPClient*, e o tratamento dos dados do *backend* foi feito através da *ArduinoJson*.

Implementação do *firmware*

O *firmware* foi desenvolvido em linguagem *C++*, utilizando a plataforma *Arduino IDE*. Sua estrutura é dividida nas funções principais *setup()* e *loop()*. Na função *setup()*, executada apenas uma vez ao iniciar o sistema, o *ESP32* solicita via comunicação serial as credenciais de rede e do usuário. Em seguida, realiza a conexão com a rede *Wi-Fi* e envia os dados de autenticação para o *backend*, por meio do *endpoint* de *login*. Após a autenticação, o sistema solicita o identificador da sala e obtém suas informações por meio de uma requisição *HTTP* ao *endpoint* de salas. Os dados retornados — como nome da sala, sensores e atuadores — são armazenados localmente no *SPIFFS*, junto com as credenciais *Wi-Fi* e o *token* de autenticação do usuário. A partir dessas informações, o sistema configura automaticamente os pinos de entrada e saída do microcontrolador, ajustando o comportamento do dispositivo de acordo com o ambiente em que será utilizado. Por fim, o módulo estabelece conexão com o *broker MQTT* e se inscreve nos tópicos necessários.

Na função *loop()*, o sistema executa continuamente a coleta de dados dos sensores e a publicação dessas informações via *MQTT*. Também é responsável por monitorar mensagens recebidas, executando comandos de controle, como o acionamento de atuadores.

RESULTADOS E DISCUSSÃO

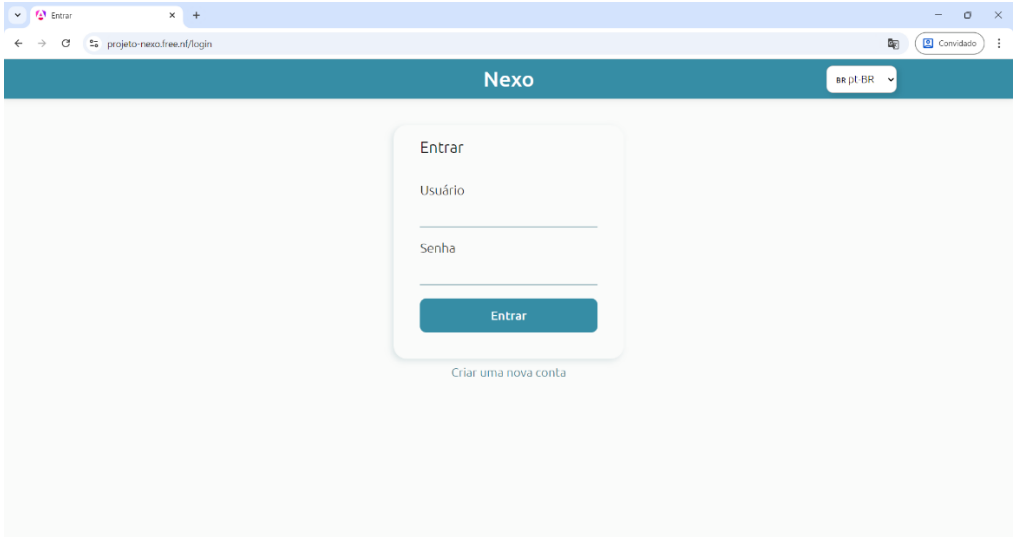
Como resultado da aplicação dessa metodologia, obteve-se um sistema acessível pelo navegador de *internet* que permite ao usuário criar uma conta, criar ambientes virtuais personalizados de laboratórios remotos e interagir com eles em conjunto com outras pessoas,

monitorando as leituras dos sensores, controlando os atuadores, acompanhando as imagens do laboratório em tempo real e realizando experimentos com coleta de amostras automáticas.

Fluxo de usuário

Para usar o sistema, o usuário primeiramente acessa o endereço *URL* da aplicação pelo navegador, sendo recebido por uma tela de autenticação (figura 2).

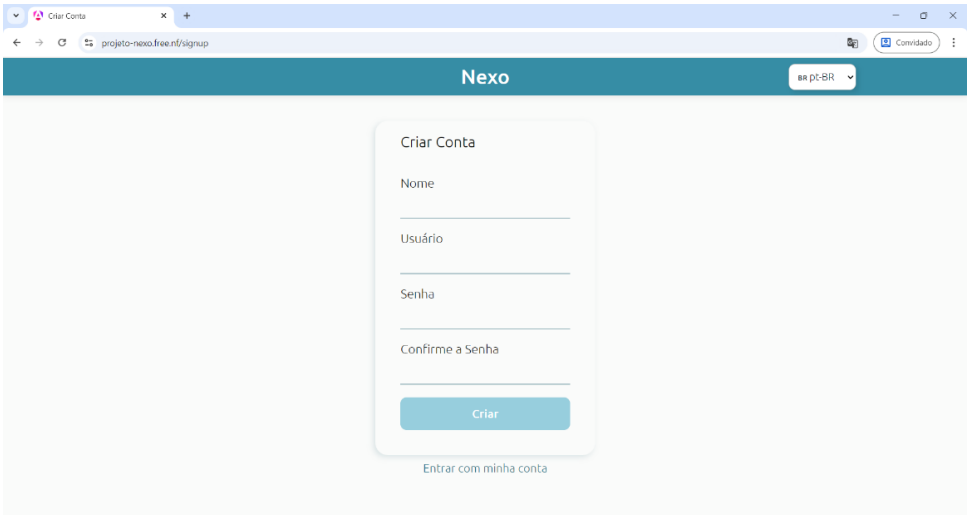
Figura 2: Tela de Autenticação



Fonte: Do próprio autor.

Caso o usuário não tenha uma conta, pode criar uma acessando a tela de criação de contas (figura 3).

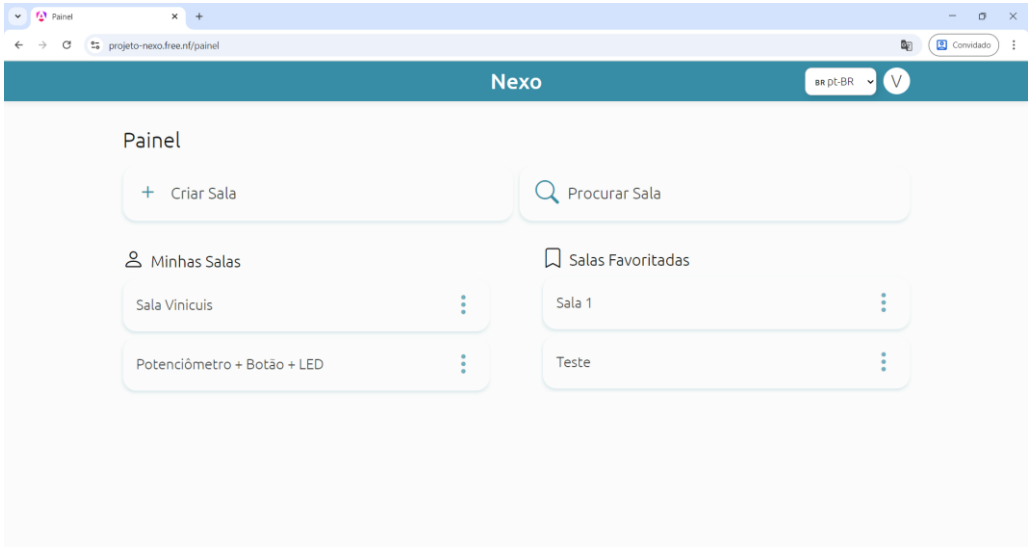
Figura 3: Tela de Criação de Contas



Fonte: Do próprio autor.

Com uma conta criada, o usuário é recebido pelo painel principal (figura 4).

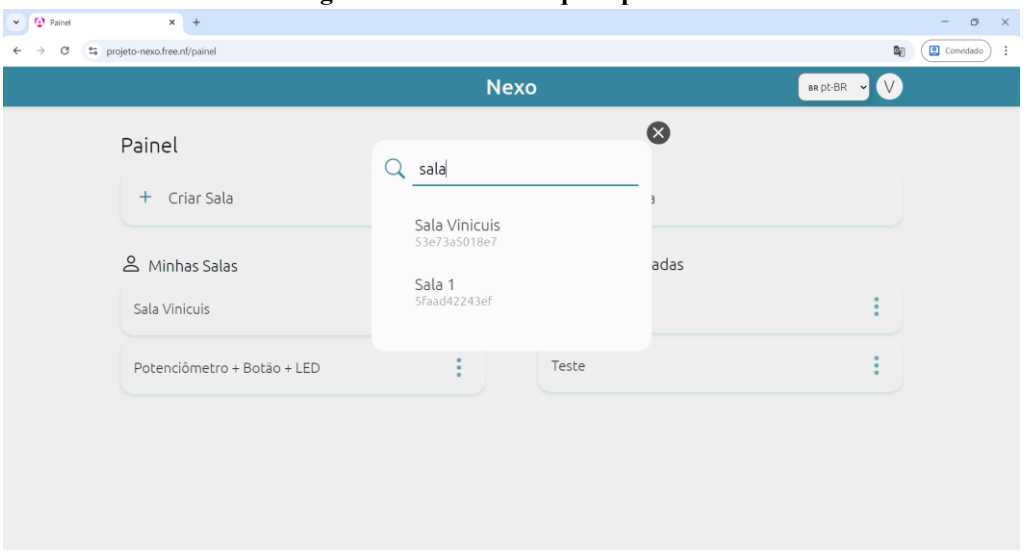
Figura 4: Painel Principal



Fonte: Do próprio autor.

No topo da página, é possível acessar a tela de criação de salas ou abrir a caixa de pesquisa por salas criadas por outros usuários (figura 5). Logo abaixo, são listadas as salas criadas anteriormente pelo usuário ou salvas na sua lista de favoritos.

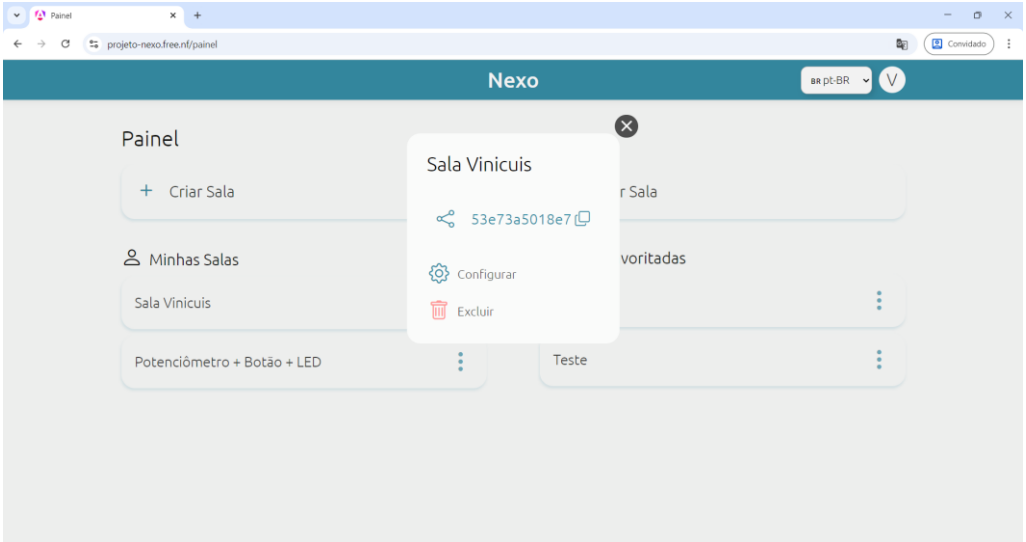
Figura 5: Caixa de Pesquisa por Salas



Fonte: Do próprio autor.

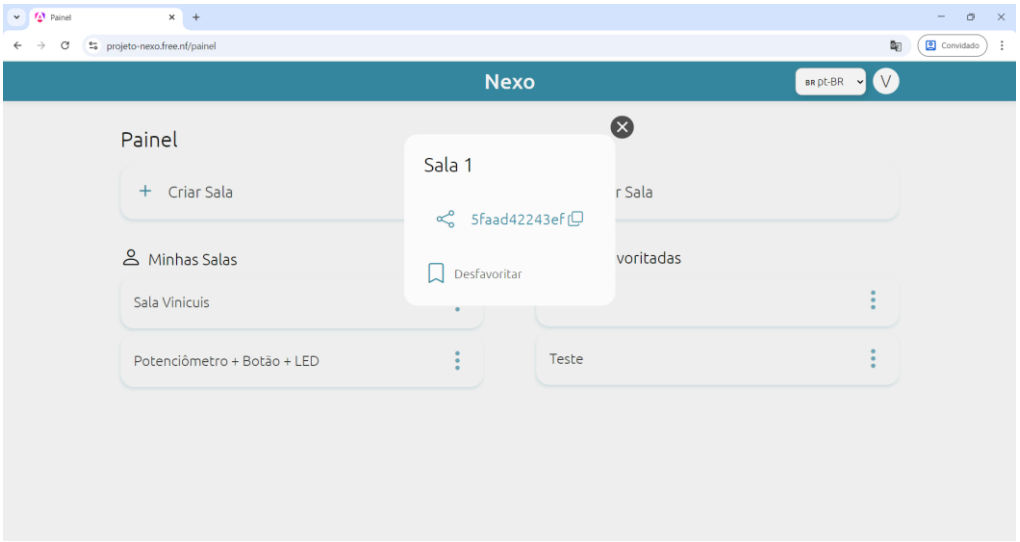
Cada um dos itens de qualquer uma das listas redireciona o usuário para o respectivo ambiente remoto ao ser clicado e inclui um botão do lado direito para abrir um painel com opções adicionais para o item (figuras 6 e 7).

Figura 6: Opções Adicionais para Item da Lista "Minhas Salas"



Fonte: Do próprio autor.

Figura 7: Opções Adicionais para Item da Lista "Salas Favoritadas"

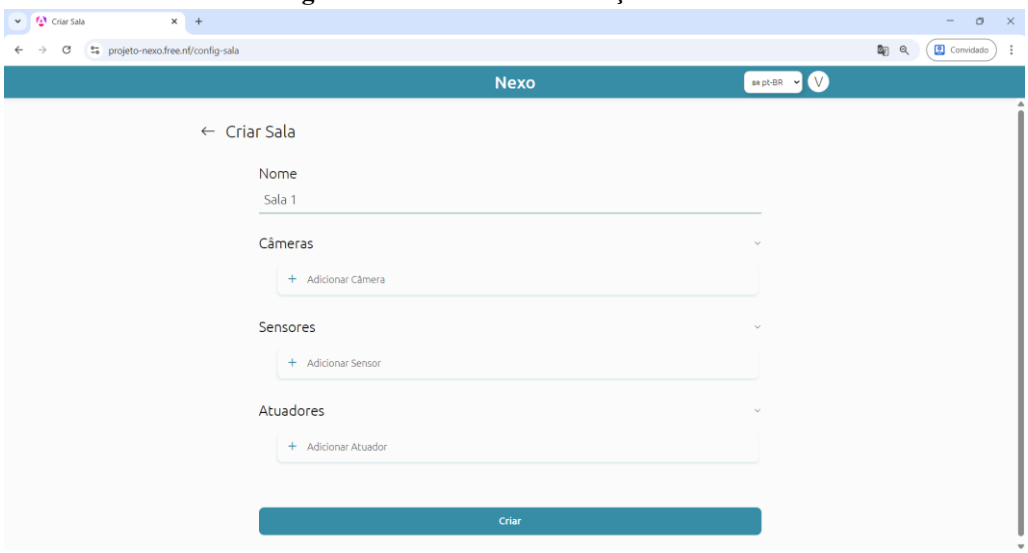


Fonte: Do próprio autor.

As caixas de diálogo para as opções adicionais dos itens das listas de salas (figuras 6 e 7) exibem o identificador único da sala. Esse é o valor da chave primária na tabela *salas* do banco de dados. Com isso, é possível encontrar a sala exata desejada, mesmo que outras tenham o mesmo nome. Além disso, esse valor é requisitado pelo sistema embarcado ao configurá-lo pela primeira vez, para que possa se conectar ao ambiente virtual correto.

Ao clicar no botão “Criar Sala”, o usuário será redirecionado para uma tela em que pode definir o nome do novo ambiente virtual, a quantidade de câmeras que deseja instalar no laboratório, atribuir um nome a cada uma delas e definir os sensores e atuadores usados. (Figura 8).

Figura 8: Formulário de Criação de Salas

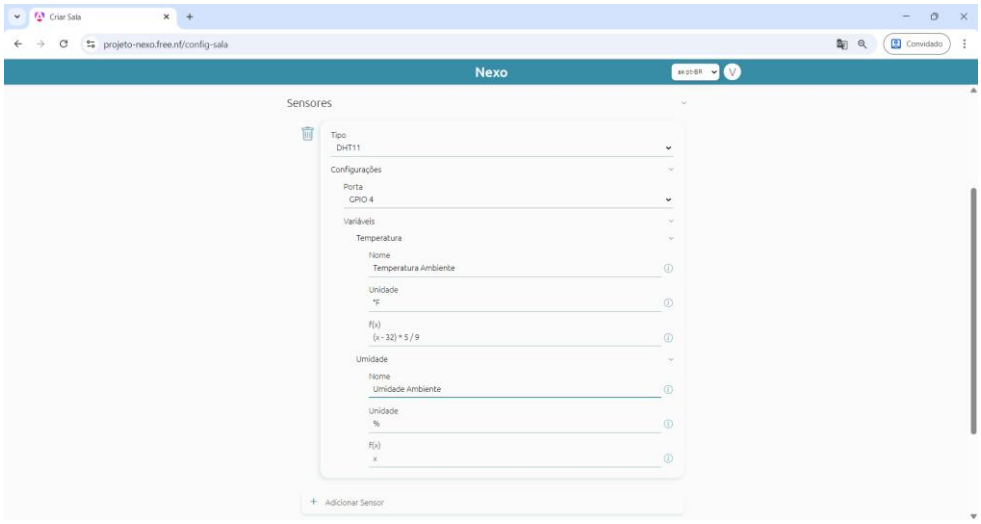


Fonte: Do próprio autor.

Para cada uma dos dispositivos – sensores e atuadores – podem ser definidas configurações específicas como porta *GPIO* do *ESP-32* conectada, módulo usado dentre as opções disponíveis, como *DHT11* (mede temperatura e umidade) e *HX711* (mede forças aplicadas, em quilogramas). Até o momento, estão implementados suporte a sensores e atuadores digitais e analógicos e módulo *DHT11*, com as outras opções na interface gráfica apenas demonstrando a flexibilidade da aplicação para possíveis incrementações futuras.

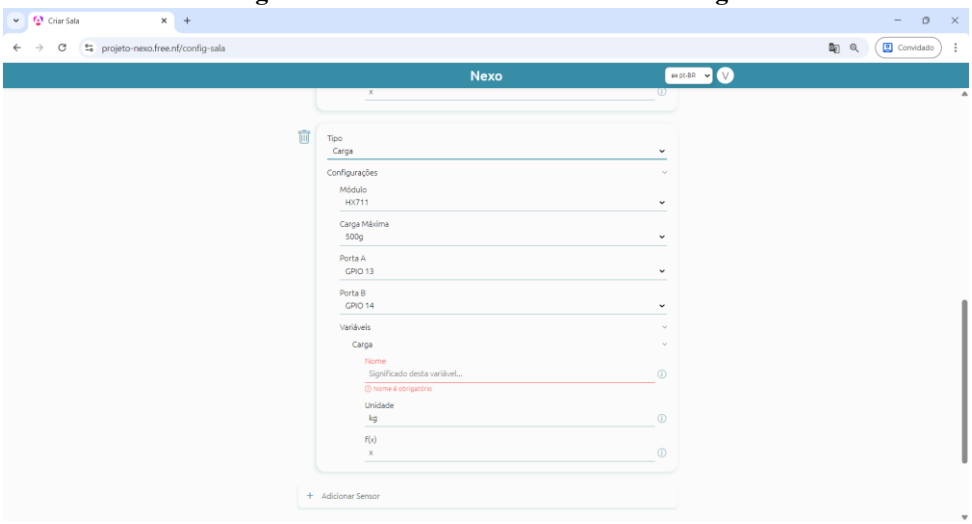
Além disso, para cada uma das variáveis associadas ao dispositivo – medida por um sensor ou controlada por um atuador – pode ser atribuído um nome, unidade de medida e função de conversão de unidades (figura 9 e 10).

Figura 9: Adicionando um Sensor de Temperatura e Umidade Atmosférica



Fonte: Do próprio autor.

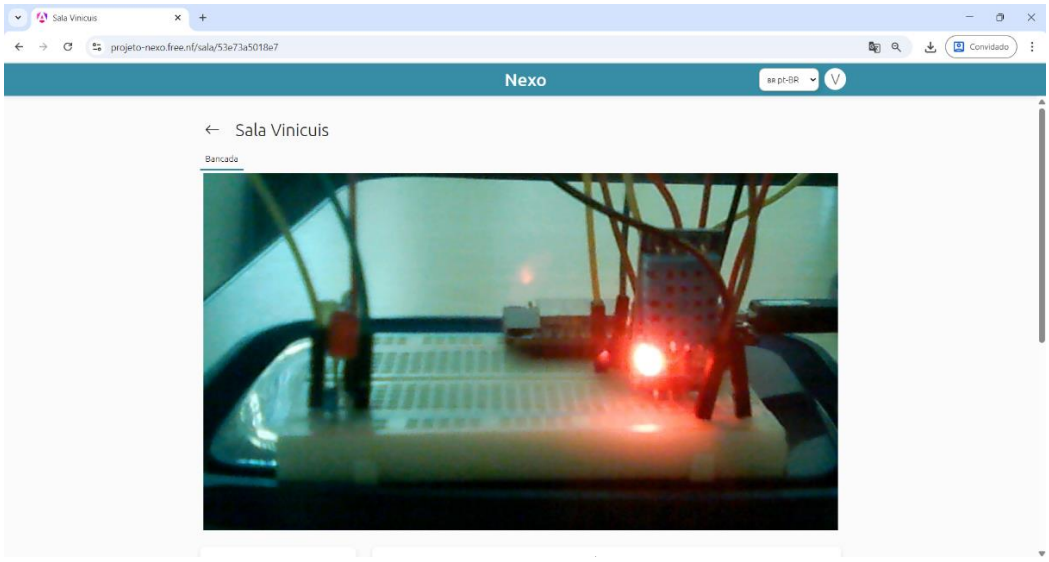
Figura 10: Adicionando um Sensor de Carga



Fonte: Do próprio autor.

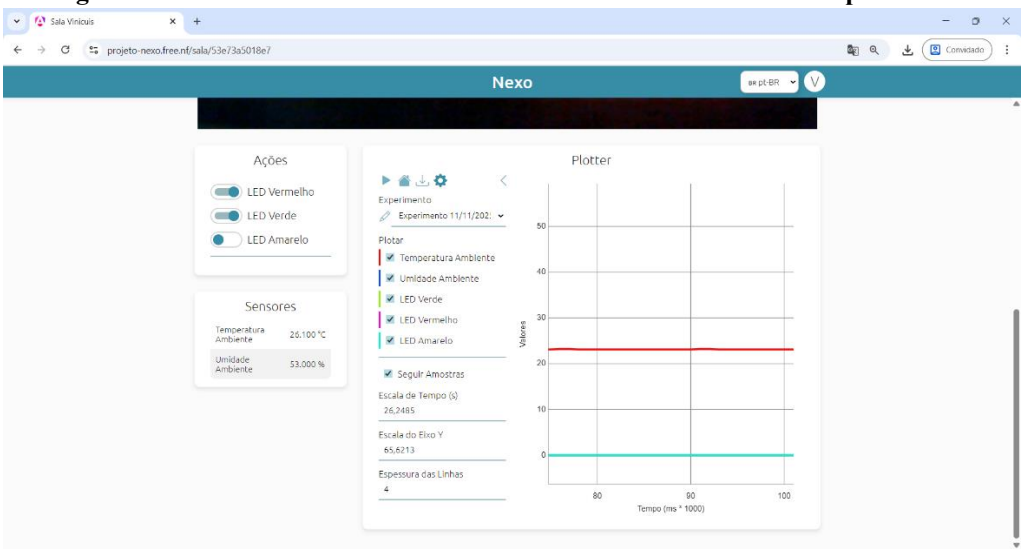
Com uma sala criada, o usuário pode então compartilhar seu identificador com outros usuários, configurar seu dispositivo embarcado e acessá-la a partir do painel inicial. Isso o levará para uma tela que inclui um tocador de vídeo, um painel de controle dos atuadores, um painel de monitoramento dos sensores e um gerenciador de experimentos com visualizador gráfico embutido denominado "*plotter*" (figuras 11 e 12).

Figura 11: Reprodutor de Vídeo



Fonte: Do próprio autor.

Figura 12: Painéis de Controle e Monitoramento e Gerenciador de Experimentos



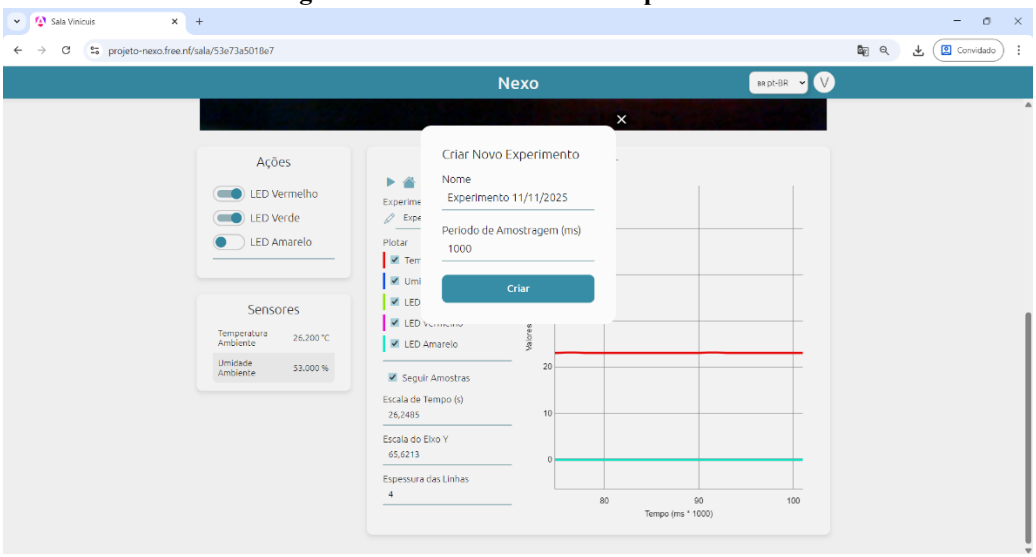
Fonte: Do próprio autor.

Toda a interface é gerada dinamicamente de acordo com as definições do usuário no momento da criação da sala. Os nomes das câmeras são listados horizontalmente acima do tocador de vídeo, alternando a visualização ao serem selecionados. Os nomes das variáveis associados a atuadores são listados no painel de controle "Ações", acompanhados de interruptores ou deslizadores, para valores digitais ou analógicos, respectivamente. Interações dos usuários com esses elementos são traduzidas em mensagens *MQTT* publicadas no tópico "*set*", definindo o novo valor desejado.

O *plotter* acompanha uma aba à esquerda que permite ao usuário, dentre outras coisas, iniciar novos experimentos, alternar para a visualização de experimentos anteriores, desabilitar a exibição de qualquer um dos gráficos das variáveis e salvar as amostras coletadas em formato "*Comma Separated Values*" (*CSV*).

Ao iniciar um experimento, é enviado ao embarcado via *MQTT* uma mensagem solicitando o início da coleta de amostras. Os valores amostrados pelo embarcado são enviados via *MQTT* assim que são coletados e acumulados na memória local, sendo periodicamente salvos no banco de dados em lotes e apagados da memória local. Caso o usuário se conecte a uma sala com um experimento em andamento, o *plotter* automaticamente alterna para a sua visualização e encarrega-se de buscar as amostras anteriores na memória local do embarcado com a mensagem *MQTT* "*sinc-exp*" ou do banco de dados, caso sejam muito antigas. A figura 13 representa a interface de criação de experimentos.

Figura 13: Criando um Novo Experimento

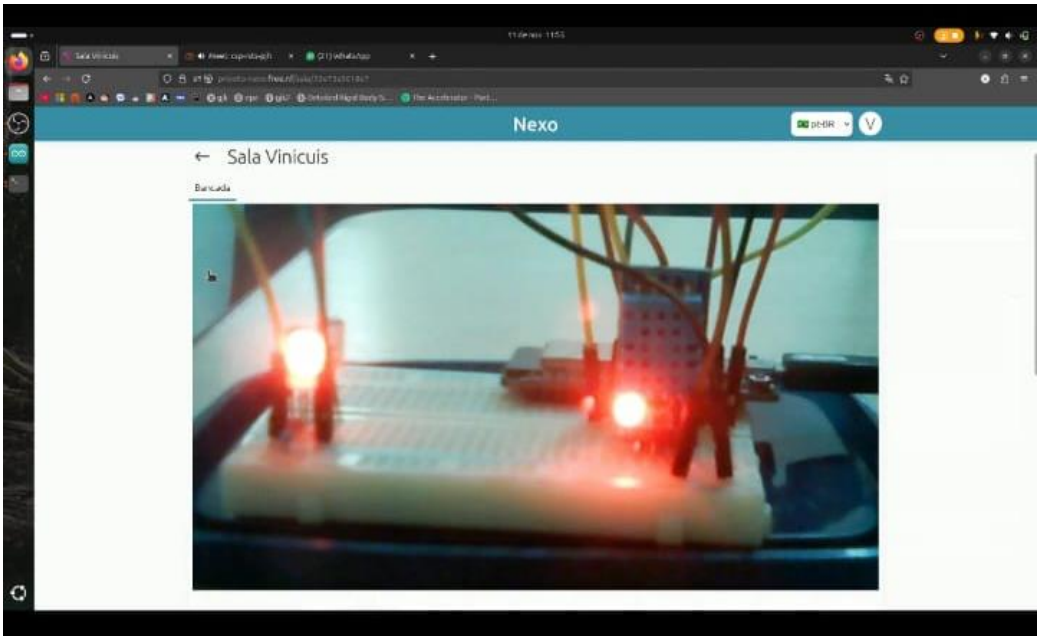


Fonte: Do próprio autor.

A plataforma foi disponibilizada publicamente, em um endereço na internet que executa os *scripts* em *typescript* do *backend* e hospeda o banco de dados *MySQL*. Foi realizada uma série de experimentos e testes, durante os quais foram tirados os *prints* que constituem as figuras anteriores. Tais testes foram essenciais para a manutenção e ajustes no código afim de diminuir e minimizar ao máximo os erros, dentre funções não executando e seções não sendo acessadas ao serem chamadas. Feitos os devidos ajustes, foi feita uma gravação de um teste para registro.

O primeiro teste público foi feito pelo *Google Meet*, junto ao orientador, após as devidas precauções e procedimentos. (Figura 14).

Figura 14: Reunião de teste



Fonte: Do próprio autor.

Durante o teste foi aberta uma *stream*, enquanto o proprietário da mesma compartilhava a tela. A *stream* foi mantida por um bom período de tempo, comprovando a capacidade da plataforma de conectar com o aplicativo e com a câmera, enquanto outros acessam a sala e realizar experimentos e amostras através do *plotter*.

Dentre os principais pontos fracos destaca-se a segurança dos dados, uma vez que as informações de conexão armazenadas no banco de dados são facilmente acessíveis através de ferramentas de desenvolvedor, abrindo brecha para possíveis vazamentos de imagem e comunicação direta com as placas de outros.

Estimativa de custos

Estimar os custos de uma implantação real desse sistema, pronta para atender usuários, envolve fazer um estudo comparativo das diversas opções de hospedagem do servidor, planos do serviço *IVS* e do provedor de *broker MQTT*, além de uma análise dos padrões de uso dos usuários, como a complexidade das salas criadas – número de câmeras, sensores e atuadores – quantidade de experimentos realizados e amostras coletadas. Assim, para simplificar a análise e tirar conclusões mais elucidativas, foi tomado como caso de uso uma implementação inicial, com requisitos razoáveis.

Tabela 1 – Relação: tabela; quantidades de linhas inseridas; tamanho em disco utilizado**.**

	Tabela	Quantidade de Linhas	Tamanho em Disco (MB)
	usuarios	10000	1,5
	salas	1000	1,5
	experimentos	10000	7,5
	amostras	1000000	75,7
Total			86,2

Fonte: Próprio autor.

A tabela 1 relaciona os requisitos de espaço de armazenamento para as principais tabelas do sistema que têm tendência a ocupar mais espaço. Os testes foram realizados com inserção de dados fictícios realistas para salas com 7 variáveis. A complexidade de memória é menor que *O(n)*, dado que o cálculo inclui o tamanho do índice da tabela, que cresce com complexidade menor que *O(n)* (ORACLE, 2025).

Assim, para pequeno porte, é possível explorar alternativas gratuitas de hospedagem *MySQL* como *Aiven*, que oferece 1*GB* de armazenamento (AIVEN, 2025). De forma similar, é possível encontrar serviços gratuitos de hospedagem de *site* como *InfinityFree* (INFINITYFREE, 2025) ou *GitHub Pages* (GITHUB, 2025), além de serviços de hospedagem de *API* com *NodeJs* com cotas razoáveis de utilização gratuita como *AWS Lambda* (AMAZON WEB SERVICES, 2025) ou *Cloud Run* (GOOGLE, 2025). Os serviços de hospedagem de *broker MQTT* também dispõem de alternativas com cotas gratuitas, como *HiveMQ* (HIVEMQ, 2025).

Dessa forma os custos iniciais seriam compostos principalmente pelo serviço de transmissão de vídeo *IVS* e implementação dos laboratórios físicos. Os custos de transmissão de vídeo podem ser estimados com base nas horas de transmissão e número de espectadores (AMAZON WEB SERVICES, 2025). Utilizando o canal de tipo básico, resolução 480p e com 8 espectadores, o custo estimado é de US$0,49 por hora de transmissão, sendo US$0,20 relativos à ingestão de vídeo e, o restante, linearmente proporcional ao número de espectadores. Já a implementação dos laboratórios pressupõe no mínimo um microcontrolador *ESP32*, que pode ser encontrado por valores em torno de US$5,33.

Tabela 2 – Custos de implementação e mantimento do sistema

Categoria	Custo
Banco de dados	Gratuito
Hospedagem de site	Gratuito
Hospedagem de *API*	Gratuito
Broker MQTT	Gratuito
Transmissão de vídeo	(US$0,20 + US$0,036 × espectadores) × hora × transmissões
Microcontrolador	US$5,33

Fonte: Próprio autor.

CONCLUSÃO

Foi implementado com sucesso o sistema de acesso remoto, a criação das salas e as transmissões são estáveis e consistentes, o que comprova sua viabilidade no âmbito técnico. Quanto ao aspecto econômico, apesar das limitações o projeto pôde ser desenvolvido e executado sem grandes problemas e com custo mínimo, mostrando ser possível e efetivo seu uso para o objetivo principal de acesso e transmissão remota. A plataforma se mostrou uma ótima solução para testes extensos de projetos de engenharia, com as mais diversas funções e configurações feitas para os mais diversos casos. A prioridade principal foi manter o custo baixo e o sistema funcional.

Há várias outras funcionalidades que poderiam ser adicionadas conforme necessidade, como opções novas para outros sensores e atuadores, funções de monitorar o *status* de uma sala e de registrar e exibir um histórico das salas dos usuários. Adicionalmente, adaptações e cartões informativos podem ser implementados para facilitar o entendimento de uso da plataforma. Por fim, melhoras no design da mesma para deixar o ambiente visualmente mais agradável e intuitivo. As limitações do projeto poderiam ser contornadas com o uso de credenciais, em troca de aumentar o custo do mesmo.

REFERÊNCIAS

AIVEN. Set up your free MySQL database. **Aiven**, 2025. Disponivel em: <https://aiven.io/free-mysql-database>. Acesso em: 2 dez. 2025.

AMAZON WEB SERVICES. AWS Lambda. **AWS**, 2025. Disponivel em: <https://aws.amazon.com/pt/lambda/>. Acesso em: 2 dez. 2025.

AMAZON WEB SERVICES. IVS Player SDK. **AWS Documentation**, 2025. Disponivel em: <https://docs.aws.amazon.com/ivs/latest/LowLatencyUserGuide/player.html>. Acesso em: 18 set. 2025.

AMAZON WEB SERVICES. IVS Stream Ingest | Real-Time Streaming. **AWS Documentation**, 2025. Disponivel em: <https://docs.aws.amazon.com/ivs/latest/RealTimeUserGuide/rt-stream-ingest.html>. Acesso em: 18 set. 2025.

AMAZON WEB SERVICES. Preços do Amazon Interactive Video Service. **AWS**, 2025. Disponivel em: <https://aws.amazon.com/pt/ivs/pricing/>. Acesso em: 2 dez. 2025.

AMAZON WEB SERVICES. What is Amazon IVS Real-Time Streaming? **AWS Documentation**, 2025. Disponivel em: <https://docs.aws.amazon.com/ivs/latest/RealTimeUserGuide/what-is.html>. Acesso em: 18 set. 2025.

ATMOKO, R. A. IoT real time data acquisition using MQTT protocol. **Journal of Physics: Conference Series**, Maio 2017. 012003.

BERALDO, A. L. D. S.; OLIVEIRA, T. D.; STRINGHINI, D. Laboratórios remotos e virtuais no Brasil com foco no ensino: Uma revisão sistemática da literatura. **Revista Novas Tecnologias na Educação**, Porto Alegre, 28 Julho 2021. 330-340.

BRÖLL, L. M.; HANSHANS, C.; KIMMERLE, D. IoT on an ESP32: Optimization Methods Regarding Battery Life and Write Speed to an SD-Card. **Embedded Systems Lab Publications**, 2023. Disponivel em: <https://ouci.dntb.gov.ua/en/works/4VZRLBY4/?utm_source=chatgpt.com>.

BROWN, A. S.; BADRICK, T. The next wave of innovation in laboratory automation: systems for auto-verification, quality control and specimen quality assurance. **Clinical Chemistry and Laboratory Medicine**, 2023. 37-43.

CHEW, Z. Y.; ARSHAD, M. R. A Low-Cost IoT Environmental Monitoring System Using ESP32. **Applied Mechanics and Materials**, Junho 2021.

DA SILVEIRA, R. et al. APLICAÇÃO DO PROTOCOLO MQTT PARA INTEGRAÇÃO DE DISPOSITIVOS IIOT A SISTEMAS DE AUTOMAÇÃO INDUSTRIAL EM UM AMBIENTE DE TESTES. **Revista Brasileira de Mecatrônica | Brazilian Journal of Mechatronics, v.4 n.2**, 21 dez. 2021. 01-15.

DATE, C. J. **Introdução a Sistemas de Banco de Dados**. 8. ed. São Paulo: Elsevier, 2004.

ESPRESSIF SYSTEMS. ESP32 Series Datasheet. **Espressif Systems**, 2020. Disponivel em: <https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf>. Acesso em: 21 Setembro 2025.

GAEHWILER, J. arduino-mqtt. **GitHub**, 2024. Disponivel em: <https://github.com/256dpi/arduino-mqtt>. Acesso em: 3 dez. 2025.

GITHUB. GitHub Pages na documentação do Jekyll. **Documentos do GitHub**, 2025. Disponivel em: <https://docs.github.com/pt/pages>. Acesso em: 2 dez. 2025.

GOLINELLI, M. H. D. M.; SILVA, J. B. D.; YEVSEYEVA, O. ARQUITETURA DE DISPOSITIVOS INTELIGENTES PARA LABORATÓRIOS REMOTOS: UMA REVISÃO SISTEMÁTICA. **RETEC - Revista de Tecnologias**, 2018.

GOOGLE. Funções do Cloud Run. **Google Cloud**, 2025. Disponivel em: <https://cloud.google.com/functions>. Acesso em: 2 dez. 2025.

HEUSER, C. A. **Projeto de Banco de Dados**. 7ª. ed. Porto Alegre: Clube de Autores, 2024.

HIVEMQ. Industrial AI Platform. **HiveMQ**, 2025. Disponivel em: <https://www.hivemq.com/>. Acesso em: 2 dez. 2025.

INFINITYFREE. Free Website Hosting. **InfinityFree**, 2025. Disponivel em: <https://www.infinityfree.com/>. Acesso em: 2 dez. 2025.

LIGHT, R. A. Mosquitto: server and client implementation of the MQTT protocol. **Journal of Open Source Software, v. 2, n. 13**, 17 maio 2017. 265.

LOPES, G. B.; FERNANDES JÚNIOR, R. F. **Desenvolvimento de um sistema de monitoramento e análise de dados para eficiência energética em indústrias 4.0.** Congresso Brasileiro de Automática – CBA. [S.l.]: [s.n.]. 2021.

ORACLE. 18.2.2 Space Needed for Keys. **MySQL**, 2025. Disponivel em: <https://dev.mysql.com/doc/refman/8.4/en/key-space.html>. Acesso em: 2 dez. 2025.

ROGER, M. AsyncMqttClient — Asynchronous MQTT client for ESP8266 and ESP32, 2020. Disponivel em: <https://github.com/marvinroger/async-mqtt-client>. Acesso em: 5 Outubro 2025.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. [S.l.]: GEN LTC, 2020.

TYPESCRIPT. What is TypeScript?, 2025. Disponivel em: <https://www.typescriptlang.org/>. Acesso em: 9 set. 2025.

ZHOU, Z. et al. Delay-reliability-aware protocol adaptation and quality of service guarantee for message queuing telemetry transport-empowered electric Internet of Things. **International Journal of Distributed Sensor Networks, v. 18, n. 5,** 13 maio 2022.